

Atama

Sprint 2 Retrospective

Team 20 - Atama

Emma Jiang, Pawani Surthi, Manya Gupta, Lizzy Zhou

What went well?

User story #16: As a user, I would like to upload my notes or slideshows in PDF form so that the application generates flashcards for me, using AI if enabled. Otherwise, formatted text can also be used to create flashcards.

STATUS: Completed

Comments: We managed to finish this user story. We divided the story into two halves; the first half being uploading formatted CSV and TXT files that the app would then parse and fill out a flashcard set for and the second half being that the user can upload lecture slides and the app would pass them onto Gemini for Gemini to create flashcard sets out of the material in the pdf. We can make a few improvements like allowing the user to upload multiple pdfs, and allowing the user to edit before creating and saving the flashcard set but none of them are functional requirements.

User story #38: As a user, I would like to receive desktop notifications that tell me when to start studying based on the study schedule from the preceding user story.

STATUS: Completed

Comments: We were able to completely finish this user story. The app allows the user to set both desktop and email notifications and dictate when the reminder is sent out. There was a bug where multiple emails would be sent out, but we were able to resolve it.

User story #40: As a user, I would like to set an exam date and have a countdown as a reminder.

STATUS: Completed

Comments: We were able to completely finish this user story. The user is able to create an exam where they can set when the exam is (date and time) and they are able to set whether or not they want a reminder about their exam. The reminder can be both desktop or email. There are some minor improvements that can be made where the reminder can be made recurring and maybe the amount of studying done for an exam

can be tracked. These improvements are necessary for function and are suggested for documentation and further improvement.

User story #12: As a user, I would like to access and use other users' flashcard sets to study.

STATUS: Completed

Comments: This story required the implementation of a homepage and a search bar. We also included the functionality to make individual flashcard sets private vs public here, as this would affect the search results. This was completely implemented and other user's study materials can easily be found using the search. Clicking on the items leads to the study pages so it is easy to access and start studying.

User story #8: As a user, I would like to stay signed into the application after authentication.

STATUS: Completed

Comments: In this story we implemented JWT tokens and stored user information in the httpOnly cookie to recognize users and provide user specific features. This also allows users to stay logged into the application across pages and after closing the site. This required a decent amount of work to add this to the existing codebase in the backend and to the api calls in the frontend.

User story #42: As a user, I would like a built-in timer to utilize study time management methods, such as Pomodoro.

STATUS: Completed

Comments: The timer built for the application is a lightweight frontend popup that can also be used to implement study methods like Pomodoro, as it has a break timer. The popup can easily be dragged across all pages and is consistent between tabs. The timer also continues even if it is closed, for ease of use.

User story #27: As a user, I would like to manually join (Purdue) specific classes in my school for the semester.

STATUS: Completed

Comments: This user story involved creating the university page for users. The page automatically populates depending on the user's email domain. From there, the user can see all of the courses of the university, search for courses, and join or leave the courses. Something we might still want to address is how to mass-add classes to the database.

User story #31: As a user, I would like to search for content for specific classes at my school.

STATUS: Completed

Comments: This user story involved creating a search bar for the users to search for specific classes. It was implemented and performs correctly.

User story #32: As a user, I would like to upload, access, and download study materials in the shared drive for my specific class.

STATUS: Completed

Comments: This user story involved creating the course pages for specific university courses. On a course page, users can upload and download PDFs, as well as add study materials from their personal libraries. The functionality is fully implemented, but some more visual UI aspects and sorting capabilities would benefit the usability.

User story #44: As a user, I would like to be able to migrate study materials from other platforms (e.g., Quizlet flashcards) to this platform.

STATUS: Completed

Comments: This story was relatively painfree to implement. There are 7 delimiters that users can choose from, which allow users to import from a large variety of other study apps or formats. The functionality also correctly errs and even shows the line where the parsing error occurs.

User story #13: As a user, I would like to rate others' flashcard sets with a star system.

STATUS: Completed

Comments: Works well with user story 14 and star ratings are stored in the backend to be pulled from to find average. Had some mishaps with pulling the info during search but was fixed.

User story #14: As a user, I would like to review others' flashcard sets with a tagging system similar to that of RateMyProfessors with tags such as "Quick to study", "All class content", "Chapter-specific", "Professor-specific", etc.

STATUS: Completed

Comments: Tags are relatively barebone and the list of predetermined tags could definitely be easily expanded. The tags are separated into negative and positive tags to make it easier for the User to find the right ones. Overall reviewing a set is very intuitive.

User story #39: As a user, I would like to observe and maintain a study streak.

STATUS: Completed

Comments: There were some problems with getting the streak to work with the database and properly show up in the UI. There were also some problems with the additional streak features (best streak and email reminder). However the story was still able to be completed by the end of the sprint.

User story #41: As a user, I would like to view the progress I have made in studying a set and how much time I have spent on it.

STATUS: Completed

Comments: This story was somewhat hard to conceptually implement as the example statistics given in the tasks did not feel intuitive and thus it was necessary to decide other statistics to display to the user about any given flashcard set. Thankfully the ones that were chosen already existed in the database and were thus easy to pull.

What did not go well?

After JWT tokens were implemented on one branch and merged into main, certain services broke because they had not been changed to work with the new authentication system. We had to spend some time fixing the errors and retesting to ensure that once-working features were working again. Ideally, we should have implemented tokens earlier or from the start to avoid this problem and frustration, especially because we knew we needed the feature.

Our Sprint 2 Demo did run into a hiccup where our grader missed us demo-ing an entire user story. Thankfully, we were given a chance to show it again, but we should have been more thorough and clear about showing each user story and acceptance criteria. We also would have benefited from practicing our presentation a few times together before the demo.

How should we improve?

In general, we were able to do a lot of manual testing as many of our features were more focused on the front end this sprint; however, we developed no automated tests. For our last sprint, we would like to develop automated tests for our own learning benefit, as well as to make testing easier for ourselves.

We would also like to be more mindful about the order in which user stories need to be implemented. For example, if user story A depends on user story B, obviously user story A should be implemented last and both stories should probably be assigned to the same person. We ran into some roadblocks where user stories depended on each other but they were assigned to different people, so one person had to wait for the other to push their changes to main before they could start working. We want to avoid dependence like that, so we will try to be more purposeful about assigning user stories and the order in which we implement them.

Finally, we would like for everyone to pull from main more consistently. This will help us work with the most updated code without conflicts, which we sometimes ran into after merges during this sprint. It is easy to get absorbed in your own branch, so we need to remember to stay up to date with main and avoid breaking our code.